1. Type of columns in the data frame
2. Unique values
3. Most frequent values
4. Descriptive statistics like mean, mode, standard deviation, sum, median absolute deviation, coefficient of variation, kurtosis, skewness
5. Quantile statistics like minimum value, Q1, median, Q3, maximum range, interquartile range
6. Histogram
7. Correlations highlighting highly correlated variables; Spearman, Pearson and Kendall matrices
8. Missing values matrix, count, heatmap and dendrogram of missing values
9. File and image analysis extract file sizes, creation dates and dimensions, and scans for truncated images or those containing EXIF information
10. Text analysis — learn about categories (uppercase, space), scripts (Latin, Cyrillic) and blocks (ASCII) of text data

Streamlit

Model deployment is the process of taking the trained model to the world and allowing everyone to use it. Traditionally, the trained model is exposed as an API by using a Python Web framework like Flask or Django. This requires a steep learning curve and can be a huge burden for the data scientist who already has a lot of work pressure. Streamlit comes to the rescue, as it removes the front-end and design barriers for data nerds.

Streamlit is an open source Python library that helps to build machine learning Web apps in no time. With just a few lines of code, we are able to build an ML application and deploy it. Using Streamlit's invite-only sharing feature, we can effortlessly share, manage and collaborate apps with team members. It is also compatible with other data science libraries like PyTorch, TensorFlow, Scikit, Plotly, Matplotlib and Bokeh.

To install Streamlit using pip package manager, type:

```
pip install streamlit
```

Streamlit provides simple APIs, which help to create many widgets. These are not limited to adding maps, charts and interactivity into apps with checkboxes, buttons and sliders, but can do much more. Given below are some of the APIs.

1 Display text:

- *st.text('Fixed width text')*
- *st.markdown('_Markdown_')*
- *st.latex(r''' e^{i\pi} + 1 = 0 ''')*
- *st.write('Most objects')*
- *st.title('My title')*
- *st.header('My header')*
- *st.subheader('My sub')*
- *st.code('for i in range(8): foo()')*

2 Display data:

- *st.dataframe(my_dataframe)*
- *st.table(data.iloc[0:10])*
- *st.json({'foo':'bar','fu':'ba'})*

3 Display charts:

- *st.line_chart(data)*
- *st.area_chart(data)*
- *st.bar_chart(data)*
- *st.pyplot(fig)*
- *st.altair_chart(data)*
- *st.vega_lite_chart(data)*
- *st.plotly_chart(data)*
- *st.bokeh_chart(data)*
- *st.pydeck_chart(data)*
- *st.deck_gl_chart(data)*
- *st.graphviz_chart(data)*
- *st.map(data)*

4 Display media:

- *st.image('./header.png')*
- *st.audio(data)*
- *st.video(data)*

5 Display interactive widgets:

- *st.button('Hit me')*
- *st.checkbox('Check me out')*
- *st.radio('Radio', [1,2,3])*
- *st.selectbox('Select', [1,2,3])*
- *st.multiselect('Multiselect', [1,2,3])*
- *st.slider('Slide me', min_value=0, max_value=10)*
- *st.select_slider('Slide to select', options=[1,'2'])*
- *st.text_input('Enter some text')*
- *st.number_input('Enter a number')*
- *st.text_area('Area for textual entry')*
- *st.date_input('Date input')*
- *st.time_input('Time entry')*
- *st.file_uploader('File uploader')*
- *st.color_picker('Pick a color')*

We can now write a few lines of code in Python to build a beautiful ML Web app with no need of back-end code, defining routes, handling HTTP requests, connecting a front-end, and writing HTML, CSS and JavaScript.

Let's take an example. We have a regression model to predict the marks of a student based on two independent variables — 'No. of hours studied' and 'No. of classes attended'.